Probing for Arithmetic Errors in Language Models

Yucheng Sun* **Alessandro Stolfo*** **Mrinmaya Sachan**
ETH Zürich
{yucsun, stolfoa}@ethz.ch

arXiv:2507.12379v1 [cs.CL] 16 Jul 2025

Abstract

We investigate whether internal activations in language models can be used to detect arithmetic errors. Starting with a controlled setting of 3-digit addition, we show that simple probes can accurately decode both the model's predicted output and the correct answer from hidden states, regardless of whether the model's output is correct. Building on this, we train lightweight error detectors that predict model correctness with over 90% accuracy. We then extend our analysis to structured chain-of-thought traces on addition-only GSM8K problems and find that probes trained on simple arithmetic generalize well to this more complex setting, revealing consistent internal representations. Finally, we demonstrate that these probes can guide selective re-prompting of erroneous reasoning steps, improving task accuracy with minimal disruption to correct outputs. Our findings suggest that arithmetic errors can be anticipated from internal activations alone, and that simple probes offer a viable path toward lightweight model self-correction.

1 Introduction

Large language models have recently shown strong performance on mathematical problem solving and reasoning (OpenAI, 2024; Team, 2024a; Shao et al., 2024; DeepSeek-AI, 2025, *inter alia*), making mathematical ability an increasingly central axis for benchmarking model capabilities (Glazer et al., 2024). This surge in capability has sparked growing interest in understanding how these models internally process numerical information and execute arithmetic reasoning (Nanda et al., 2023; Stolfo et al., 2023a; Hanna et al., 2023; Zhong et al., 2023; Zhou et al., 2024b; Nikankin et al., 2025; Lindsey et al., 2025).

In particular, recent studies have investigated how pre-trained language models represent numerical quantities (Levy and Geva, 2025; Kantamneni and Tegmark, 2025; Zhu et al., 2025). However, despite increasing insight into the structure of these representations, their practical use for improving model behavior remains limited. At the same time, analyzing hidden representations has proven useful for identifying model failures and hallucinations in other domains (Kadavath et al., 2022; Azaria and Mitchell, 2023; Yuksekgonul et al., 2024; Chen et al., 2024; Orgad et al., 2025).

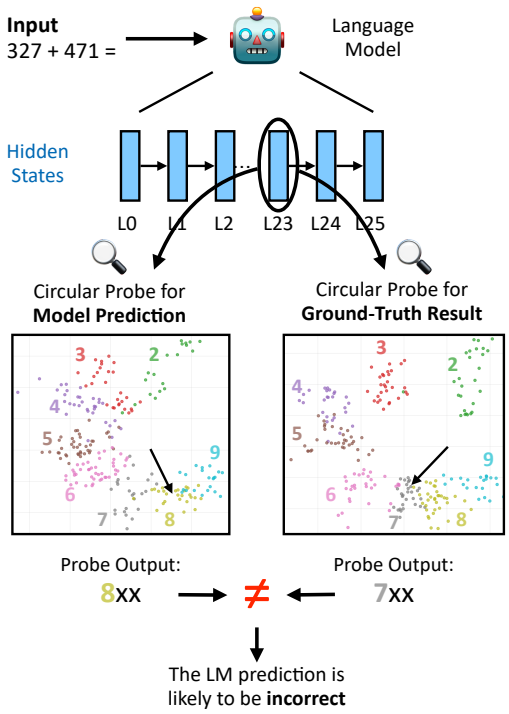

Figure 1: **Detecting Arithmetic Errors from Hidden States.** We investigate whether internal activations in a language model reveal when its arithmetic predictions are incorrect. We train simple probes to decode both the model's output and the correct answer. The probes' output can serve as a reliable signal of model error.

Motivated by these findings, we investigate whether internal model activations can be leveraged to detect arithmetic errors in language models. We begin with a simple setting: the model is prompted with 3-digit addition queries (e.g.,

*Equal contribution.

"327+471"), and we analyze its hidden activations using simple probes inspired by representational and circuit-level findings. We show that not only it is possible to recover the language model's prediction, but also to predict the ground-truth result from the model's hidden states, independently of whether the model's output is correct. Motivated by this observation, we adapt the probing method to predict whether the model's final answer will be correct (Figure 1), achieving over 90% accuracy in predicting model correctness on a balanced dataset.

We then extend this analysis to a more complex setting, where the model is asked to solve math word problems only requiring addition (Cobbe et al., 2021) using a structured chain-of-thought (CoT) format (Wei et al., 2022), in which intermediate steps are expressed as equations (e.g., <a+b=c>). Remarkably, we find that the same probes trained on simple arithmetic queries can be applied directly to this setting, maintaining over 80% accuracy in detecting whether the model is producing correct intermediate results.

Finally, we explore the practical benefits of these probes by using them as weak oracles to identify potentially erroneous steps within the model's reasoning traces. We design a re-prompting mechanism that selectively revisits flagged steps, leading to an overall correction of up to 11% of the wrong reasoning steps, without compromising any of the correct ones. These results suggest that internal representations of numerical quantities are robust across contexts, and can be exploited to detect and correct model errors with lightweight probing methods.

2 Probing for Numerical Representations

We aim to understand how information about arithmetic operations is internally represented in a language model. To this end, we consider a controlled setting in which the model is prompted with simple 3-digit addition queries and analyze its hidden activations at the point just before it produces an output. Specifically, we ask whether numerical quantities are encoded in a form that can be recovered by lightweight probes. We begin with a representation analysis (§2.1), then introduce a set of probing methods to decode this information (§2.2).

2.1 Representation Analysis

Prior work has shown that language models represent periodic concepts such as the days of the week in circular forms (Engels et al., 2025). In the mathematical domain, similar findings suggest that numerical quantities can be encoded in circular (Levy and Geva, 2025), linear (Zhu et al., 2025), or hybrid helix-like (Kantamneni and Tegmark, 2025) geometries. To better understand how numerical information is encoded in our setting, we conduct a representation analysis on the instruction-tuned version of Gemma 2 2B (Team, 2024b), focusing on its behavior when solving 3-digit addition problems (e.g., "327+471").[1]

We synthetically generate arithmetic queries by sampling 800 pairs of operands $(a, b) \in \{100, \ldots, 999\}^2$, such that $a + b < 1000$. Then, we feed the model the prompt "a+b=", and examine the model's internal activations at the position of the equals sign ("="). This token position immediately precedes the model's output and is expected to carry information about the computed result (Stolfo et al., 2023a; Nikankin et al., 2025).

We apply principal component analysis (PCA) to the residual stream activations at this position across all layers of the model, highlighting in different colors the hidden states associated with different hundreds digits in the ground-truth result. We observe that as depth increases, the representations of individual digits become increasingly structured and separable. In particular, deeper layers exhibit clearer clustering by digit number, with the top principal components revealing a circular layout similar to those observed in prior studies.

To illustrate this progression, we selected two representative layers for visualization. As shown in Figure 2, layer 15 shows no clear digit separation, whereas layer 25 exhibits both stronger clustering and a visible circular layout. This progression supports the hypothesis that the model gradually builds a more geometric and abstract encoding of numerical concepts over its depth, and is consistent with what previous work observed.

2.2 Probing Methods

To quantitatively assess how numerical information is encoded in the model's internal representations, we design a set of lightweight probing methods. Building on the insights from our representational analysis, which indicated a circular structure in

[1]The Gemma tokenizer splits numbers into individual digits. For our experiments, we focus on predicting the first digit of the result (i.e., the hundreds place). This choice captures the majority of the model's errors in this setting (evidence for this in Appendix F). Results for a model with a different tokenization scheme (Phi-3), which confirm our findings, are reported in Appendix I.

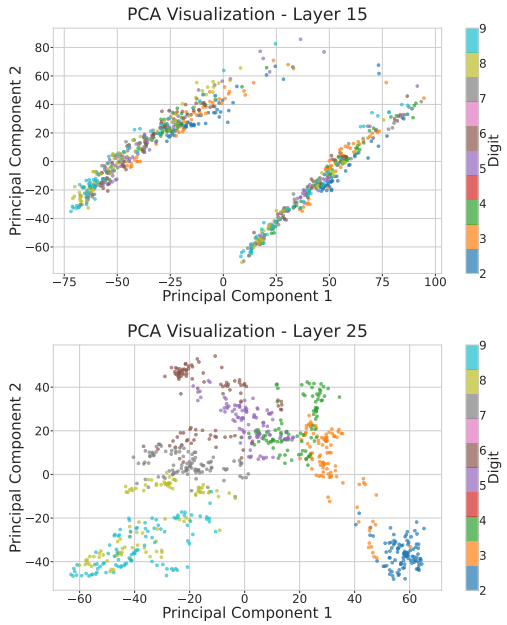

Figure 2: **PCA of Residual Stream Activations.** PC projections colored by the hundreds digit of the ground-truth result. Representations become more structured with depth, showing clear digit clusters and a circular layout in deeper layers.

digit encodings at deeper layers, we treat circular probing as a natural baseline. However, we also explore alternative probing approaches to evaluate different ways of extracting numerical information from the same representations.

As in the previous section, we focus on the residual stream activations at the equals sign. Each probe is trained and evaluated independently at each layer of the model, allowing us to track how the accessibility of digit-level information evolves across the model's depth. We describe the implementation of each probe below.

Circular Probe. Denote the residual-stream hidden states of a language model at layer l by $\mathbf{x}_l \in \mathbb{R}^{d_{\text{model}}}$.[2] Let $y \in \{0, \ldots, 9\}$ be the label that the probe is tasked to predict. We design a circular probe that projects the model's hidden states onto a plane, then uses the angle that such projection point forms with the origin as its output. More formally, let the probe weights be two vectors $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^{d_{\text{model}}}$. We compute the probe's output $\hat{y}$ as

$$\theta = \operatorname{atan2}\left(\mathbf{w}_1^\top \mathbf{x}_l, \mathbf{w}_2^\top \mathbf{x}_l\right) \in [0, 2\pi), \tag{1}$$

$$\hat{y} = \theta \cdot \frac{10}{2\pi}. \tag{2}$$

The probe is trained using smooth ℓ_1 loss (Girshick, 2015) between the prediction $\hat{y}$ and the label y. The probe is optimized using the AdamW optimizer (Loshchilov and Hutter, 2019).

Linear Probe. Motivated by prior work showing that numerical quantities can be linearly decoded from language model activations (Zhu et al., 2025; Kantamneni and Tegmark, 2025), we test whether any arithmetic information in our setting can be recovered by a simple linear probe:

$$\hat{y} = \mathbf{w}^\top \mathbf{x}_l + b, \tag{3}$$

where $\mathbf{w} \in \mathbb{R}^{d_{\text{model}}}$ and $b \in \mathbb{R}$ are the probe's parameters. The probe is trained using ℓ_2 regularization.[3] At inference time, the probe's output is rounded to the nearest integer to produce a discrete prediction.

Multi-Class Logistic Regression. We also consider a variant of the linear probe that treats digit prediction as a multi-class classification problem. Specifically, we associate a distinct weight vector $\mathbf{w}_i$ with each digit $i \in \{0, \ldots, 9\}$, and compute the probe's output as the digit corresponding to the maximum logit:

$$\hat{y} = \arg\max_i \left(\mathbf{w}_i^\top \mathbf{x}_l\right). \tag{4}$$

The probe is trained using cross-entropy loss and optimized with the Adam optimizer (Kingma and Ba, 2017).

MLP Probe. Finally, we experiment with a more expressive probe based on a multi-layer perceptron (MLP). The MLP consists of a single hidden layer with ReLU activation and a hidden dimensionality of 512. As in the logistic probe, the output layer produces 10 logits corresponding to the digit $i \in \{0, \ldots, 9\}$. The prediction is given by:

$$\hat{y} = \arg\max_i \left(\mathbf{W}_2^\top \operatorname{ReLU}\left(\mathbf{W}_1^\top \mathbf{x}_l + \mathbf{b}_1\right) + \mathbf{b}_2\right),$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times 512}$, $\mathbf{W}_2 \in \mathbb{R}^{512 \times 10}$, $\mathbf{b}_1 \in \mathbb{R}^{512}$, and $\mathbf{b}_2 \in \mathbb{R}^{10}$ are the probe's parameters. This probe is trained using cross-entropy loss and optimized with Adam.

[2] By "residual stream," we indicate per-token hidden state with dimensionality d_{model} consisting of the sum of all previous component outputs (Elhage et al., 2021).

[3] Following Zhu et al. (2025), we use regularization weight $\lambda = 0.1$.